

5

NUMBER SYSTEM

5.1. NUMBER SYSTEM

5.1.1. Base (Radix)

Total number of digit used in the system

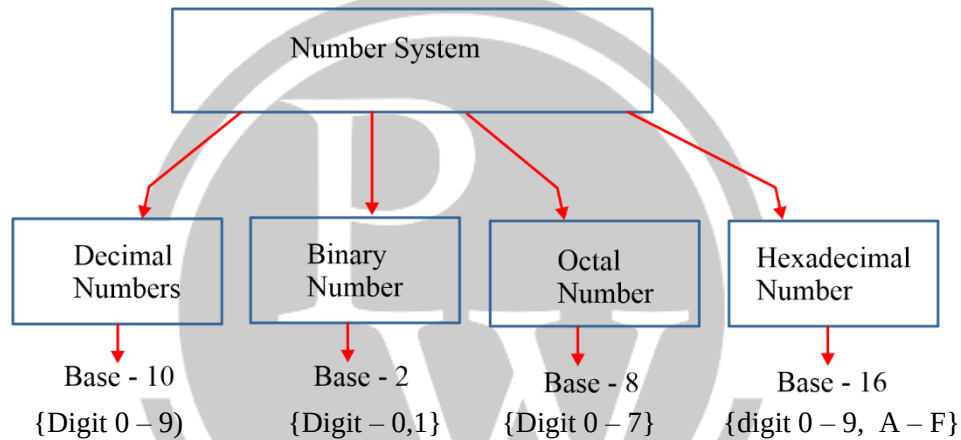


Fig. 5.1.

5.1.2. Decimal Number System

$$\begin{array}{cccccccc}
 \dots & 10^4 & 10^3 & 10^2 & 10^1 & 10^0 & 10^{-1} & 10^{-2} & 10^{-3} \dots \\
 \dots & a_4 & a_3 & a_2 & a_1 & a_0 & a_{-1} & a_{-2} & a_{-3} \dots
 \end{array}$$

$a_i \rightarrow$ Coefficient of decimal number system

$10^i \rightarrow$ Weight of decimal number system

Example: - $(501.23)_{10}$

	10^2	10^1	10^0	10^{-1}	10^{-2}
	5	0	1	2	3

Base	Digit
2	0, 1
3	0, 1, 2
4	0, 1, 2, 3
5	0, 1, 2, 3, 4
6	0, 1, 2, 3, 4, 5

7	0, 1, 2, 3, 4, 5, 6
8	0, 1, 2, 3, 4, 5, 6, 7
9	0, 1, 2, 3, 4, 5, 6, 7, 8
10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
11	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A
12	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B
13	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C
14	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D
15	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E
16	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

5.1.3 Binary Number System (Base (Radix) = 2)

...	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3} ...
...	a_4	a_3	a_2	a_1	a_0	a_{-1}	a_{-2}	a_{-3} ...

$2^i \rightarrow$ Weight of Binary number system

$a_i \rightarrow$ Coefficient of Binary number system $\{0, 1\}$

Example:-

	$(101.11)_2$
2^2	2^1 2^0 2^{-1} 2^{-2}
1	0 1 1 1

5.1.4. Octal Number System (Base (Radix) = 8)

...	8^3	8^2	8^1	8^0	8^{-1}	8^{-2}	8^{-3} ...
...	a_3	a_2	a_1	a_0	a_{-1}	a_{-2}	a_{-3} ...

$8^i \rightarrow$ Weight of Octal number system

$a_i \rightarrow$ Coefficient of Octal number system $\{0-7\}$

Example:-

	$(728.64)_8$
8^2	8^1 8^0 8^{-1} 8^{-2}
7	2 8 6 4

5.1.5 Hexadecimal Number System (Base (Radix) = 16):

...	16^3	16^2	16^1	16^0	16^{-1}	16^{-2}	16^{-3} ...
...	a_3	a_2	a_1	a_0	a_{-1}	a_{-2}	a_{-3} ...

$16^i \rightarrow$ Weight of Hexadecimal number system

$a_i \rightarrow$ Coefficient of Hexadecimal number system $\{0-9, A-F\}$

Example:

	$(A2C.F)_{16}$
16^2	16^1 16^0 16^{-1}
A	2 C F

5.1.6. In base conversion 2 key points are there:

- (A) Any base to Decimal conversion
- (B) Decimal to any other base conversion

(A) Any base to Decimal conversion:

$$(a_3 \ a_2 \ a_1 \ a_0 . a_{-1} \ a_{-2}) = (\quad)_{10}$$

$$(a_3 \times r^3 + a_2 \times r^2 + a_1 \times r^1 + a_0 \times r^0 + a_{-1} \times r^{-1} + a_{-2} \times r^{-2})_{10}$$

Case (1) : Binary to Decimal conversion

Ex. $(1011.11)_2 = (\quad)_{10}$

$$\Rightarrow [(1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) + (1 \times 2^{-1}) + (1 \times 2^{-2})]$$

$$\Rightarrow [8 + 0 + 2 + 1 + 0.5 + 0.25]_{10}$$

$$\Rightarrow (11.75)_{10}$$

Case (2) : Octal to Decimal conversion

Ex. $(721.4)_8 = (\quad)_{10}$

$$\Rightarrow [(7 \times 8^2) + (2 \times 8^1) + (1 \times 8^0) + (4 \times 8^{-1})]_{10}$$

$$\Rightarrow [448 + 16 + 1 + 0.5]_{10}$$

$$\Rightarrow (465.5)_{10}$$

Case (3) : Hexadecimal to Decimal conversion

Ex. $(A2B.C)_{16} = (\quad)_{10}$

$$\Rightarrow [(A \times 16^2) + (2 \times 16^1) + (B \times 16^0) + (C \times 16^{-1})]_{10}$$

$$\Rightarrow [(10 \times 256) + (2 \times 16) + (11 \times 1) + (12 \times 16^{-1})]_{10}$$

$$\Rightarrow [2560 + 32 + 11 + 0.75]_{10}$$

$$\Rightarrow (2603.75)_{10}$$

Case (4) : Base 5 to Decimal conversion

Ex. $(432.22)_5 = (\quad)_{10}$

$$\Rightarrow [(4 \times 5^2) + (3 \times 5^1) + (2 \times 5^0) + (2 \times 5^{-1}) + (2 \times 5^{-2})]_{10}$$

$$\Rightarrow [100 + 15 + 2 + 0.4 + 0.08]_{10}$$

$$\Rightarrow (117.48)_{10}$$

(B) Decimal to any other Base conversion

$$(a_3 \ a_2 \ a_1 \ a_0 . a_{-1} \ a_{-2} \ a_{-3})_{10} = (\quad)_r$$

Before Decimal
After Decimal

r	a_3	a_2	a_1	a_0	Remainder
					b_0
					b_1
					b_2
					b_3

$$0 \cdot a_1 a_2 a_3 \times r = x_0 \cdot x_1 x_2$$

$$0 \cdot x_1 x_2 \times r = x_1 \cdot x_3 x_4$$

$$0 \cdot x_3 x_4 \times r = x_2 \cdot x_5 x_6$$

$$(a_3 a_2 a_1 a_0 \cdot a_1 a_2 a_3)_{10} = (b_3 b_2 b_1 b_0 \cdot x_0 x_1 x_2)_r$$

Case (1) : Decimal to Binary Base conversion.

Ex.

$(19.75)_{10} = ()_2$

Before Decimal	R	After Decimal
2 19	1	$0.75 \times 2 = 1.5$ 1
2 9	1	$0.5 \times 2 = 1.0$ 1
2 4	0	
2 2	0	
1 1	1	

$(19.75)_{10} = (10011.11)_2$

Case (2) : Decimal to Octal Base conversion.

Ex.

$(210.23)_{10} = ()_8$

Before Decimal	Remainder	After Decimal
8 210	2	$0.23 \times 8 = 1.84$ 1
8 26	2	$0.84 \times 8 = 6.72$ 6
3 3	3	$0.72 \times 8 = 5.76$ 5

$(210.23)_{10} = (322.165)_8$

Case (3) : Decimal to Hexadecimal Base conversion.

Ex.

$(1228.55)_{10} = ()_{16}$

Before Decimal	Remainder	After Decimal
16 1228	12 (C)	$0.55 \times 16 = 8.8$ 8
16 76	12 (C)	$0.8 \times 16 = 12.8$ 12(C)
4 4	4	

$(1228.56)_{10} = (4CC.8C)_{16}$

5.2. Some Special Case

Case (1): Binary to Octal base conversion

Example: $(10110111)_2 = (\quad)_8$

Octal $\rightarrow$ means base 8

$$8 = 2^3$$

Every three digits of binary represent one digit of octal

010 110 111

2 6 7

Hence $(10110111)_2 = (267)_8$

Case (2): Binary to Hexadecimal base conversion

Example: $(1011011)_2 = (\quad)_{16}$

Hexadecimal $\rightarrow$ means base 16

$$16 = 2^4$$

Every four digits of binary represent one digit of Hexadecimal.

0101 1011

5 11(B)

Hence $(1011011)_2 = (5B)_{16}$

5.1.2. BCD (Binary Coded Decimal)

- In this each digit of the decimal number is represented by its four-bit binary equivalent. It is also called natural BCD or 8421 code. It is weighted code.
- Excess – 3 Code:** This is a non weighted binary code used for decimal digits. Its code assignment is obtained from the corresponding value of BCD after the addition of 3.
- BCO (Binary Coded Octal):** In this each digit of the Octal number is represented by its three-bit binary equivalent.
- BCH (Binary Coded Hexadecimal):** In this each digit of the hexadecimal number is represented by its four bit binary equivalent.

Decimal Digits	BCD 8421	Excess – 3	Octal digits	BCO	Hexadecimal Digits	BCH
0	0000	0011	0	000	0	0000
1	0001	0100	1	001	1	0001
2	0010	0101	2	010	2	0010
3	0011	0110	3	011	3	0011
4	0100	0111	4	100	4	0100
5	0101	1000	5	101	5	0101
6	0110	1001	6	110	6	0110
7	0111	1010	7	111	7	0111
8	1000	1011			8	1000



9	1001	1100			9	1001
					A	1010
					B	1011
					C	1100
					D	1101
					E	1110
					F	1111

Don't care values or unused states in BCD code are 1010, 1011, 1100, 1101, 1110, 1111.

Don't care values or unused states in excess – 3 code are 0000, 0001, 0010, 1101, 1110, 1111.

The binary equivalent of a given decimal number is not equivalent to its BCD value.

Example: $25_{10} = 11001_2$.

The BCD equivalent of decimal number 25 = 00100101 from the above example the BCD value of a given decimal number is not equivalent to its straight binary value.

The BCO (Binary Coded Octal) value of a given Octal number is exactly equal to its straight binary value.

Example: $25_8 = 21_{10} = 010101_2$

The BCO Value of 25_8 is 010101.

From the above example, the BCO value of a given Octal number is same as binary equivalent of the same number.

The BCH (Binary Coded Hexadecimal) value of a given hexadecimal number is exactly equal to its straight binary.

Example: $25_{16} = 37_{10} = 100101_2$

The BCH value of hexadecimal number $25_{16} = 00100101$.

From this example the above statement is true.

	Binary	Octal	Decimal	Hexadecimal
Complement	$r = 2$	$r = 8$	$r = 10$	$r = 16$
$(r - 1)$'s	1's	7's	9's	15's
r 's Complement	2's	8's	10's	16's

Example: Add the two Binary numbers 101101_2 .

```

Augend 101101
addend 100111
      1111
-----
Sum 1010100

```

Example: Subtract the Binary number 100111_2 from 101101_2 .

```

Minuend : 101101
Subtracted: 100111
-----
Difference: 000110

```

Example: Multiple the Binary number 1011_2 from 101_2 .

$$\begin{array}{r}
 \text{Multiplicand: } 1011 \\
 \text{Multiplier: } \quad X101 \\
 \hline
 1011 \\
 0000 \\
 1011 \\
 + \\
 \hline
 \text{Product: } 110111
 \end{array}$$

While storing the signed binary numbers in the internal registers of a digital computer} most significant bit position is always reserved for sign bit and the remaining bits are used for magnitude.

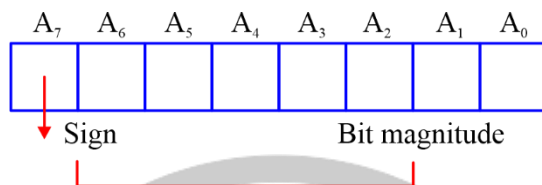


Fig. 5.2.

When the binary number is positive, the sign is represented by '0'. When the number is negative, the sign is represented by '1'.

5.2.2. Fixed-Point Representation and Floating-Point Representation;

The representation of the decimal point (ordinary point) in a register is complicated by the fact that it is characterized by a position between two flip- flops in the register.

There are two ways of specifying the position of the decimal point in a register.

- (1) Fixed Point and
- (2) Floating Point.

The fixed point method assumes that the decimal point (or binary point) is always fixed in one position. The two positions most widely used are (1) a decimal point in the extreme left of the register to make the stored number a fraction, and (2) a decimal point in the extreme right of the register to make the stored number an integer.



The floating-point representation uses a second register to store a number that designates the position of the decimal point in the first register.

Positive numbers are stored in the registers of digital computer in sign magnitude form only.

Negative number can be represented in one of three possible ways.

1. Signed – magnitude representation.
2. Signed – 1's complement representation.
3. Signed – 2's complement representation.

Example: +9

–9

Signed – magnitude 0 0001001 (a) 1 000 1001 signed – magnitude

(b) 1 111 0110 signed – 1's complement

(c) 1 111 0111 signed – 2's complement

The 2's complement of a given binary number can be formed by leaving all least significant zeros and the first non-zero digit unchanged, and then replacing 1's by 0's and 0's by 1's in all other higher significant digits.

Example: The 2's complement of 10011000_2 is 01101000 .

Subtraction using 2's complement: Represent the negative number in signed 2's complement form, add the two numbers, including their sign bit, and discard any carry out of the most significant bit.

Since negative numbers are represented in 2's complement form, negative results also obtained in signed 2's complement form.

Example: 1's complement:

+ 6 0000110	– 6 1111001	+ 6 0000110	– 6 1111001
+ 9 0001001	+ 9 0001001	– 9 1110110	– 9 1110110
+ 15 0001111	+3 (i) 0000010	– 3 1111100	– 15 (1) 1101111
	Carry + 1		Carry + 1
	+ 3 0000011		1110000
	carry		carry

The advantage of signed 2's complement representation over the signed 1's complement form (and the signed – magnitude form) is that it contains only one type of zero.

The general form of floating – point number is mr^e . Where M = Mantissa, r = base, e = exponent.

Example: $+0.3574 \times 10^5$.

The mantissa can be a fixed point fraction or fixed point integer.

Normalization: Getting non-zero digit in the most significant digit position of the mantissa is called Normalization.

- If the floating point number is normalized, more number of significant digits can be stored, as a result accuracy can be improved.
- A zero cannot be normalized because it does not contain a non-zero digit. The hexadecimal code is widely used in digital systems because it is very convenient to enter binary data in a digital system using hexcode.
- The parity of a digital word is used for detecting error in digital transmission. Hollerith code is used for punched card data.
- In weighted codes, each position of the number has specific weight. The decimal value of a weighted code number is the algebraic sum of the weights of those positions in which 1's appears.
- Most frequently used weighted codes are 8421, 2421 code, 5211 code and 8421 code.
- **Reflective Code:** A code is called reflective or self-complementing, if the code for 9 is the complement for the code for 0, code for 8 is the complement from 1 and so on. 2421, 842'1', 5211 are examples for reflected codes.
- **Sequential Code:** A code is called sequential, if each successive code-is one binary number greater than its preceding code.

Example: 8421

